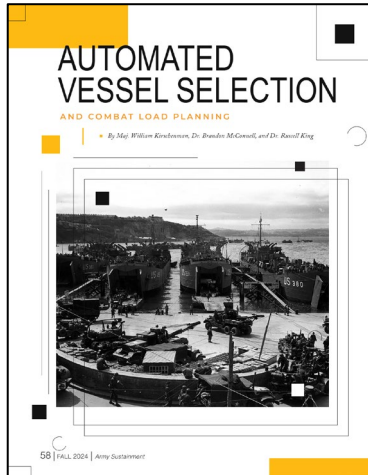




Automating Military Load Planning: A Prioritized 2-D Bin Packing Approach



93rd MORS Symposium
WG 16: Strategic Deployment and Distribution
June 9-12, 2025

MAJ Will Kirschenman

Operations Research Ph.D. Candidate
Department of Industrial and Systems Engineering, North Carolina State University

Purpose: Introduce a novel optimization framework that automates and enhances military combat load planning by systematically integrating multi-level tactical priorities and critical operational constraints, addressing gaps in current methodologies.

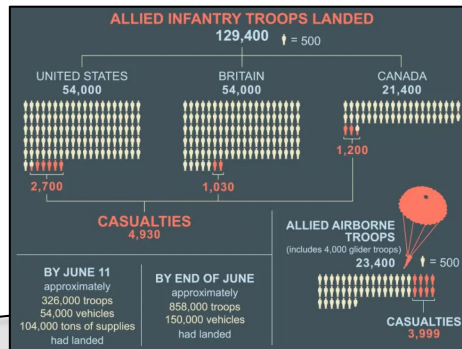
Agenda:

- Introduction.
- Baseline Model.
- Literature Review and Model Enhancements.
- Solution Techniques.
- Experimentation and Initial Results.
- Conclusion.

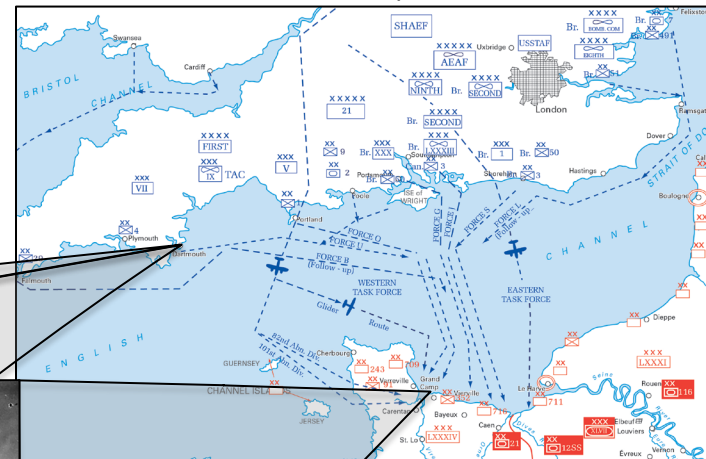
Normandy Invasion: June 6, 1944

- Initial detailed planning started in March 1943.
- Combat loading commenced in April 1944.
- Over 1,200 landing craft were loaded in 14 ports across southern England, each meticulously scheduled to avoid congestion on June 6, 1944.

APPROXIMATE NUMBER OF VEHICLES USED		
3,000 landing craft	2,500 other ships	13,000 aircraft
	500 naval vessels	
	20,000 land vehicles	



Allied Invasion Force and German Dispositions, Northern France, 1944.



Other Examples

- Incheon Landing.
- Operation Cartwheel (Rabaul).
- Operation Husky (Invasion of Sicily).
- Operation Iceberg (Battle of Okinawa).

What if we need to do this again?

Motivation

- **Tactical Sequencing (Order of March).** Equipment must be accessible and off-loading in the specific sequence required by the ground tactical plan.
- **Unit Integrity / Group Cohesion.** Military units are designed to fight as cohesive elements.
- **Asset Prioritization.** Certain assets may have higher intrinsic priority based on role and require preferential placement.
- **Speed of Off-Load.** In contested environments, the time spent off-loading is a period of high vulnerability.



1ID Artillery Equipment Loading on LSTs at Brixham, England, on June 1, 1944.



Off-loading equipment on Omaha Beach in days following June 6, 1944.

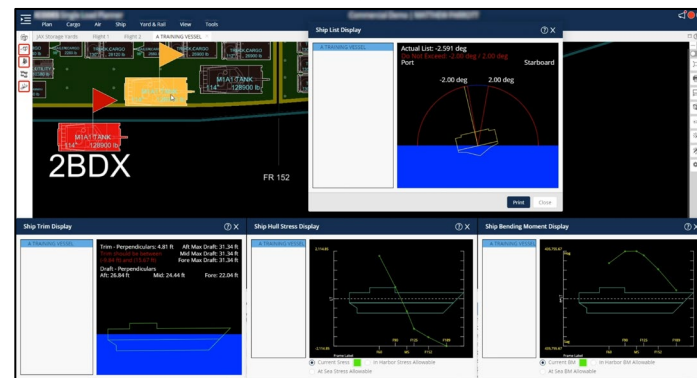
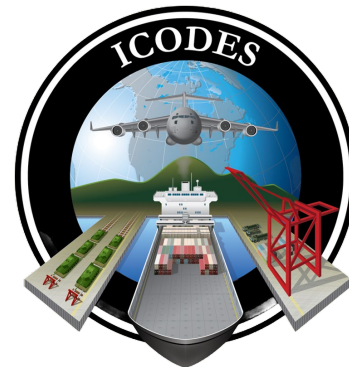
Introduction

Combat Loading Context:

- **Application:** Prioritized loading of a single vessel.
- **Goal:** Enable rapid, effective off-loading in contested environments.

Role of ICODES (Integrated Computerized Deployment System):

- **Purpose:** DoD standard system for load planning (Sea, Air, Land). Improves efficiency, safety, visibility.
- **Architecture:** Ontology-based, agent-driven system interconnecting data.
 - Uses shared data libraries (equipment, vessels).
 - Software “agents” collaborate (Stow, Stability, Hazards, etc.).
- **Key Functions:** Performs automated feasibility checks:
 - Vessel Trim & Stability.
 - HAZMAT Segregation.
 - Cargo Accessibility and Clearances.
- **Limitations:** Achieving *tactical priorities* often requires manual adjustments (“User Stow”) beyond automated checks/basic “Auto Stow.”



ICODES Single Load Planner module.

Methodology

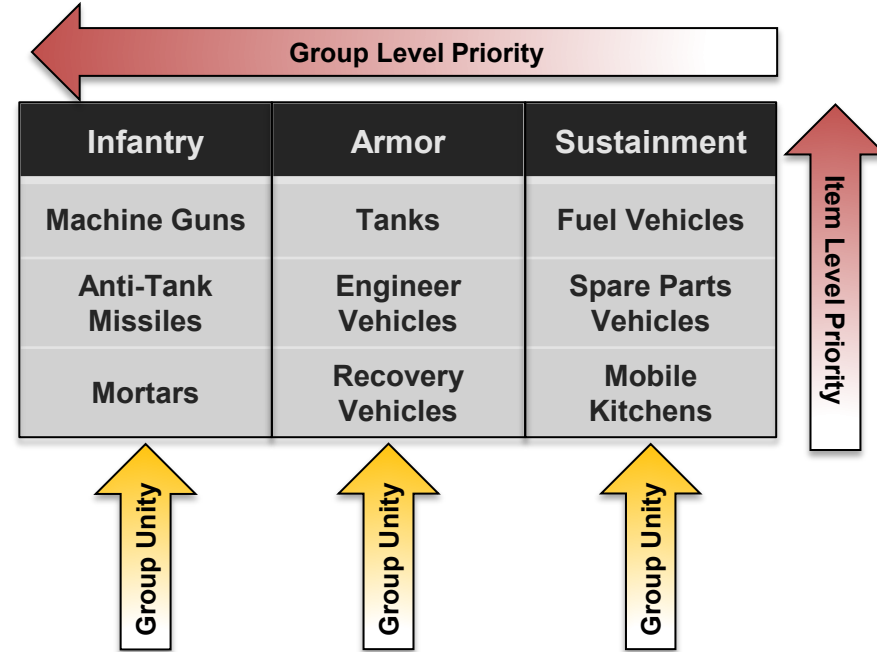
Formulation

Approach: Mixed-Integer Linear Program (MILP) based on efficient BPP formulations (Fontaine and Minner, 2023).

- Use a **Prioritization Matrix** ($\pi_{i,k}$) for spatial preferences:
 - Diagonal** ($\pi_{i,k}$ for $i = k$): Priority weight for placing item i close to the bin access point (x^o, y^o) . *Higher value = stronger pull to access point.*
 - Off-Diagonal** ($\pi_{i,k}$ for $i < k$): Priority weight for placing items i and k (often same group) close to each other (cohesion). *Higher value = stronger pull together.*
- Optimize item placement by minimizing a **Weighted-Distance Objective Function** (rectilinear distance):

$$\min \sum_{i \in I} \sum_{k \in I, i \leq k} \pi_{ik} (\rho_{ik}^x + \rho_{ik}^y)$$

- Key Constraint Categories:**
 - (1) Ensuring items fit within bin boundaries and reflect chosen orientation. (2) Preventing overlap between any pair of items. (3) Calculating rectilinear distance between item centroids and/or access point.



Solution Techniques

Solution Techniques

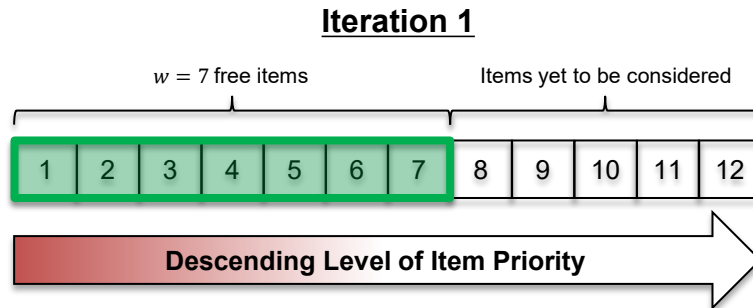
1. Single MILP Approach:

- Solve the complete MILP formulation directly using a commercial solver (e.g., Gurobi).
- Can prove optimality, but computationally expensive (often intractable for $> \sim 7$ items within reasonable time).



2. Sliding Window Decomposition:

- Break the problem into smaller, manageable subproblems based on item priority ranking.
- Iteratively solve MILPs for a small 'window' (w) of items.
- Fix the highest-priority item's position in the window, slide window forward (add next item, remove fixed item).
- *Benefit:* Avoids solving large MILP, significantly faster.
- Parameter w (window size) allows trade-off between computation time and solution quality.



Solution Techniques

Solution Techniques

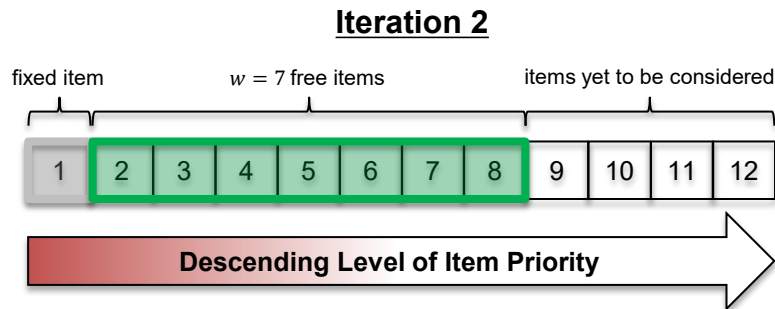
1. Single MILP Approach:

- Solve the complete MILP formulation directly using a commercial solver (e.g., Gurobi).
- Can prove optimality, but computationally expensive (often intractable for $> \sim 7$ items within reasonable time).



2. Sliding Window Decomposition:

- Break the problem into smaller, manageable subproblems based on item priority ranking.
- Iteratively solve MILPs for a small 'window' (w) of items.
- Fix the highest-priority item's position in the window, slide window forward (add next item, remove fixed item).
- *Benefit:* Avoids solving large MILP, significantly faster.
- Parameter w (window size) allows trade-off between computation time and solution quality.



Solution Techniques

Solution Techniques

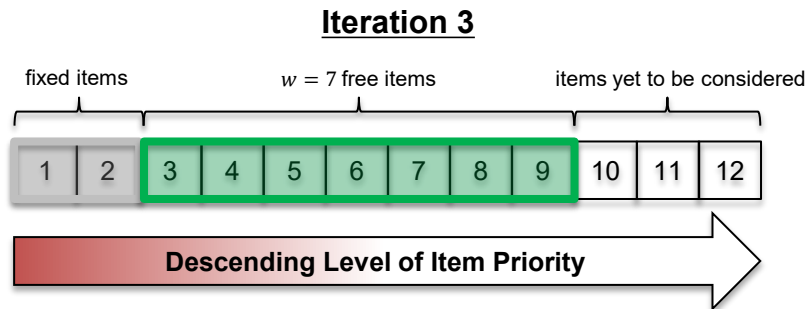
1. Single MILP Approach:

- Solve the complete MILP formulation directly using a commercial solver (e.g., Gurobi).
- Can prove optimality, but computationally expensive (often intractable for $> \sim 7$ items within reasonable time).



2. Sliding Window Decomposition:

- Break the problem into smaller, manageable subproblems based on item priority ranking.
- Iteratively solve MILPs for a small 'window' (w) of items.
- Fix the highest-priority item's position in the window, slide window forward (add next item, remove fixed item).
- *Benefit:* Avoids solving large MILP, significantly faster.
- Parameter w (window size) allows trade-off between computation time and solution quality.

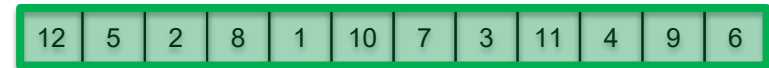


Solution Techniques

Solution Techniques

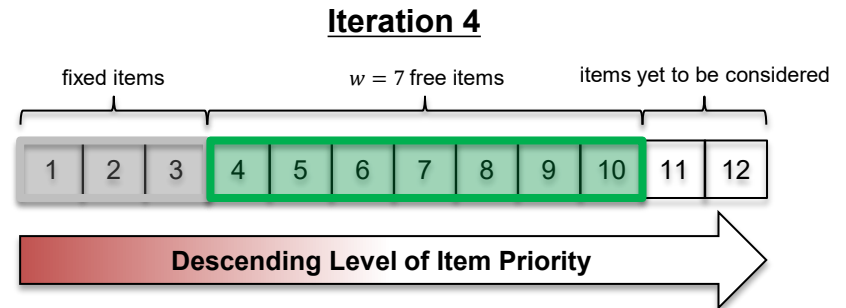
1. Single MILP Approach:

- Solve the complete MILP formulation directly using a commercial solver (e.g., Gurobi).
- Can prove optimality, but computationally expensive (often intractable for $> \sim 7$ items within reasonable time).



2. Sliding Window Decomposition:

- Break the problem into smaller, manageable subproblems based on item priority ranking.
- Iteratively solve MILPs for a small 'window' (w) of items.
- Fix the highest-priority item's position in the window, slide window forward (add next item, remove fixed item).
- *Benefit:* Avoids solving large MILP, significantly faster.
- Parameter w (window size) allows trade-off between computation time and solution quality.



Solution Techniques

Solution Techniques

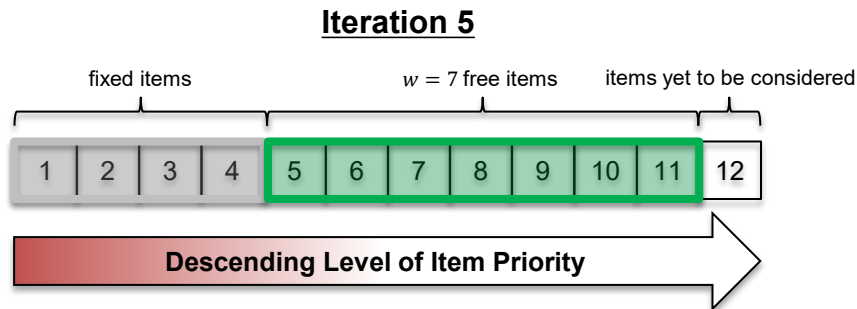
1. Single MILP Approach:

- Solve the complete MILP formulation directly using a commercial solver (e.g., Gurobi).
- Can prove optimality, but computationally expensive (often intractable for $> \sim 7$ items within reasonable time).



2. Sliding Window Decomposition:

- Break the problem into smaller, manageable subproblems based on item priority ranking.
- Iteratively solve MILPs for a small 'window' (w) of items.
- Fix the highest-priority item's position in the window, slide window forward (add next item, remove fixed item).
- *Benefit:* Avoids solving large MILP, significantly faster.
- Parameter w (window size) allows trade-off between computation time and solution quality.

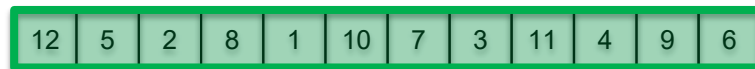


Solution Techniques

Solution Techniques

1. Single MILP Approach:

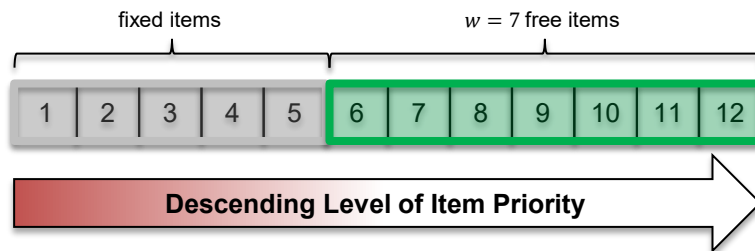
- Solve the complete MILP formulation directly using a commercial solver (e.g., Gurobi).
- Can prove optimality, but computationally expensive (often intractable for $> \sim 7$ items within reasonable time).



2. Sliding Window Decomposition:

- Break the problem into smaller, manageable subproblems based on item priority ranking.
- Iteratively solve MILPs for a small 'window' (w) of items.
- Fix the highest-priority item's position in the window, slide window forward (add next item, remove fixed item).
- *Benefit:* Avoids solving large MILP, significantly faster.
- Parameter w (window size) allows trade-off between computation time and solution quality.

Iteration 6 (Final)



Literature Review

- **Military Doctrine.**

- Combat loading principles prioritize tactical needs: sequence, cohesion, speed > space efficiency (JP 3-02, 2019).
- Historical context informs (e.g., Normandy, Inchon) modern planning challenges with large-scale combat operations (LSCO).

- **ICODES (Single Load Planner).**

- DoD standard planner; Ontology/Agent architecture integrates data (Pohl & Morosoff, 2011; Goodman & Pohl, 2003).
- Key Function: Automated checks via agents (Stability, HAZMAT, Access, etc.).
- Gap: Tactical goals often require manual planner intervention (“User Stow”) (Kirschenman et al., 2024; Longhorn et al., 2022).

- **Vessel Loading Constraints.**

- **Material Separation:** Modeled via minimum edge distances; analogous to HAZMAT rules in safe process plant layouts (Patsiatzis et al., 2004).
- **Obstacle Avoidance:** Modeled as fixed items using standard non-overlap logic (Chircop & Surendonk, 2019).
- **Load Balancing:** Critical for stability; modeled via center of gravity constraints (Trivella & Pisinger, 2016; Ramos et al., 2018; Davies & Bischoff, 1999).

Model Enhancements

Material Separation

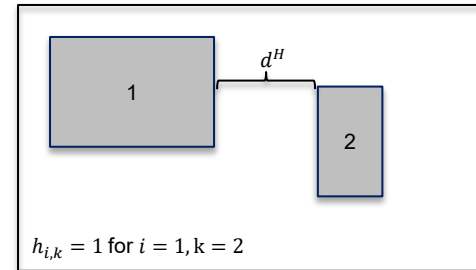
- **Purpose:** Maintain minimum edge-to-edge distance between specific item pairs (e.g., HAZMAT).
- **Approach:** Modify standard non-overlap constraints to add required separation gap.
- **Key Elements:**
 - Set of pairs requiring separation (H).
 - Minimum distance parameter (d_H), and binary indicator for pairs ($h_{i,k}$).
- **Constraints:** Integrate separation distance d_H into non-overlap logic when $h_{i,k} = 1$.

$$x_k^r \leq x_i - (h_{i,k} \cdot d^H) + (1 - a_{i,k}^x)L \quad \forall i, k \in I^E, i < k$$

$$x_i^r \leq x_k - (h_{i,k} \cdot d^H) + (1 - a_{k,i}^x)L \quad \forall i, k \in I^E, i < k$$

$$y_k^r \leq y_i - (h_{i,k} \cdot d^H) + (1 - a_{i,k}^y)W \quad \forall i, k \in I^E, i < k$$

$$y_i^r \leq y_k - (h_{i,k} \cdot d^H) + (1 - a_{k,i}^y)W \quad \forall i, k \in I^E, i < k$$



Example showing minimum material separation distance between a pair of items.



Takeaway: *This approach integrates material separation directly into the existing non-overlap logic without requiring new constraint types.*

Model Enhancements

Obstacle Avoidance

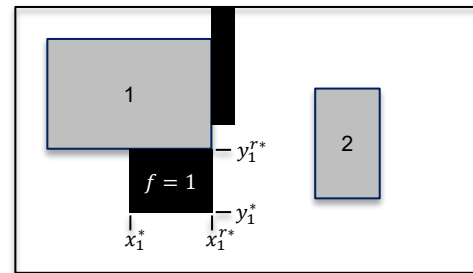
- **Purpose:** Prevent cargo overlap with fixed deck obstacles (pillars, etc.).
- **Approach:** Model obstacles as pre-positioned, immovable “items.”
- **Key Elements:**
 - Set of obstacle items (I^F).
 - Known, fixed coordinates for each obstacle: $x_f^*, y_f^*, x_f^{r*}, y_f^{r*}, \forall f \in I^F$.
- **Constraints:** Equality constraints fix obstacle positions:

$$x_f = x_f^* \quad \forall f \in I^F$$

$$y_f = y_f^* \quad \forall f \in I^F$$

$$x_f^r = x_f^{r*} \quad \forall f \in I^F$$

$$y_f^r = y_f^{r*} \quad \forall f \in I^F$$



Example showing obstacles (black fill) treated as fixed items and moveable items (gray fill) as “free” items.

★ Takeaway: *This approach integrates obstacle handling directly into the existing non-overlap logic without requiring new constraint types.*

Model Enhancements

Load Balancing

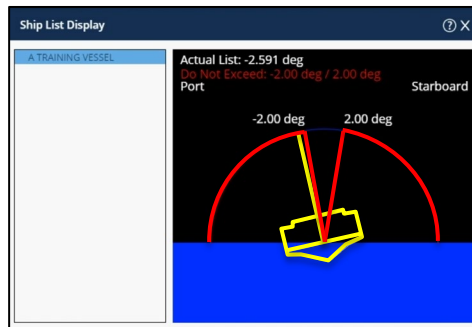
- **Purpose:** Ensure cargo Center of Gravity (CG) is within safe limits for vessel stability.
- **Approach:** Add constraints restricting the calculated overall CG of placed cargo items (I^E).
- **Key Elements:**
 - Item weights (m^i), total cargo weight ($M^E = \sum_{i \in I^E} m^i$), target CG location (B_{opt}^x, B_{opt}^y), and tolerances (δ_x, δ_y).
 - Variables for calculated CG (B^x, B^y).
- **Constraints:** Calculate actual cargo CG and enforce within tolerance box:

$$\sum_{i \in I^E} m_i \left(\frac{x_i + x_i^r}{2} \right) = M^E \cdot B^x$$

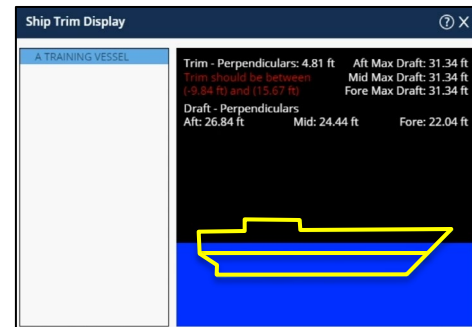
$$\sum_{i \in I^E} m_i \left(\frac{y_i + y_i^r}{2} \right) = M^E \cdot B^y$$

$$B_{opt}^x - \delta_x \leq B^x \leq B_{opt}^x + \delta_x$$

$$B_{opt}^y - \delta_y \leq B^y \leq B_{opt}^y + \delta_y$$



ICODES Single Load Planner: Ship list agent notifying the user of list (starboard-to-port) stability violations.



ICODES Single Load Planner: Ship trim agent notifying the user of trim (bow-to-stern) stability violations.



Takeaway: *These constraints ensure the center of gravity of the placed cargo items falls within the desired tolerance box.*

Solution Techniques

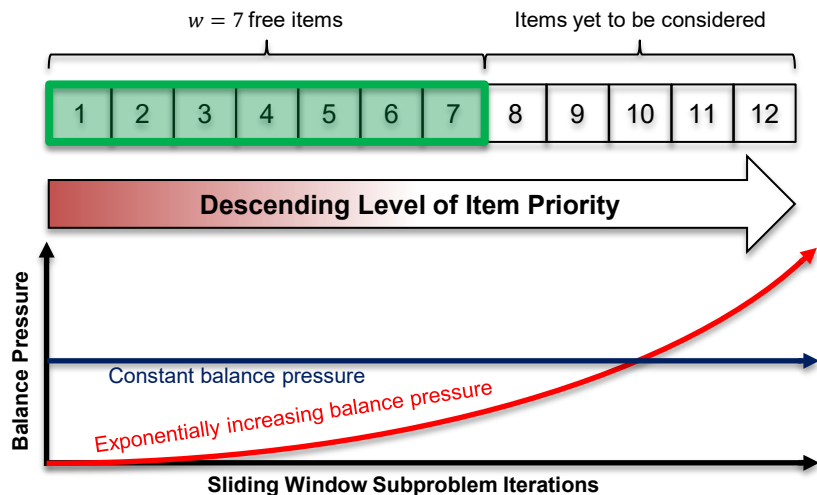
Challenge: Global Constraints in Local Decomposition

Challenge: Load balancing is a global constraint (depends on ALL items), but sliding window decomposition solves local subproblems (window w).

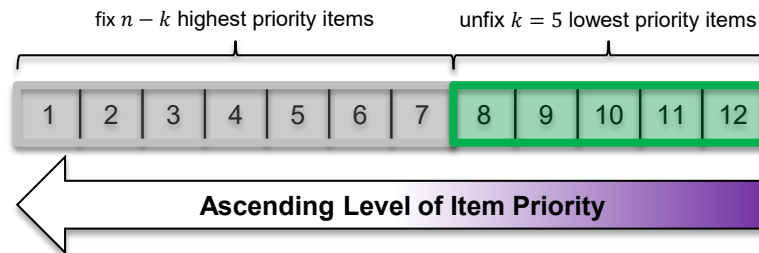
Baseline: Single MILP (with load balancing) works but has lower prioritization scores (previous work).

Goal: Adapt effective sliding window method for load balancing.

In-Stride Balancing



Post-Processing



Solution Techniques

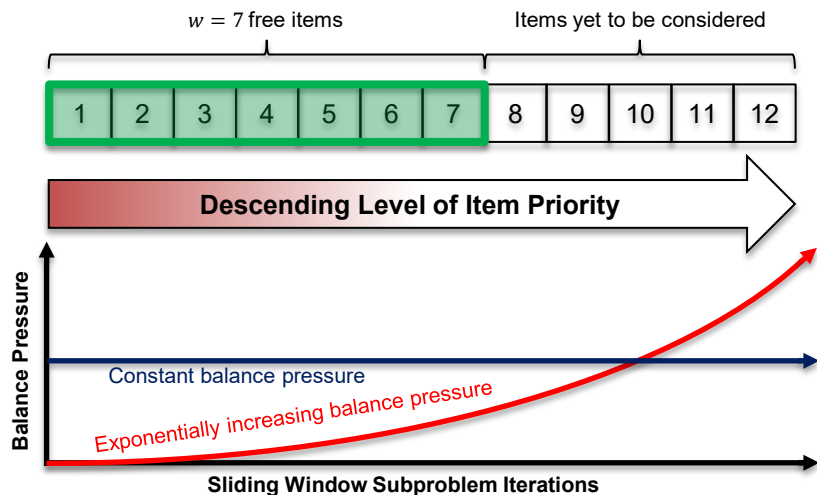
Challenge: Global Constraints in Local Decomposition

Challenge: Load balancing is a global constraint (depends on ALL items), but sliding window decomposition solves local subproblems (window w).

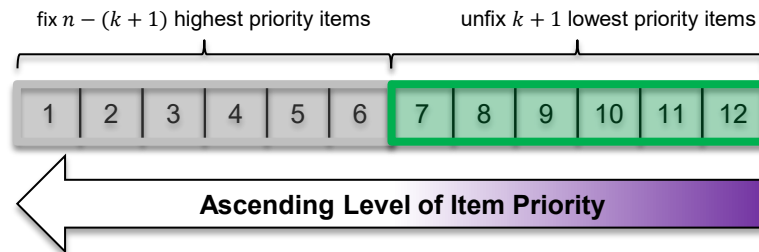
Baseline: Single MILP (with load balancing) works but has lower prioritization scores (previous work).

Goal: Adapt effective sliding window method for load balancing.

In-Stride Balancing



Post-Processing



Solution Techniques

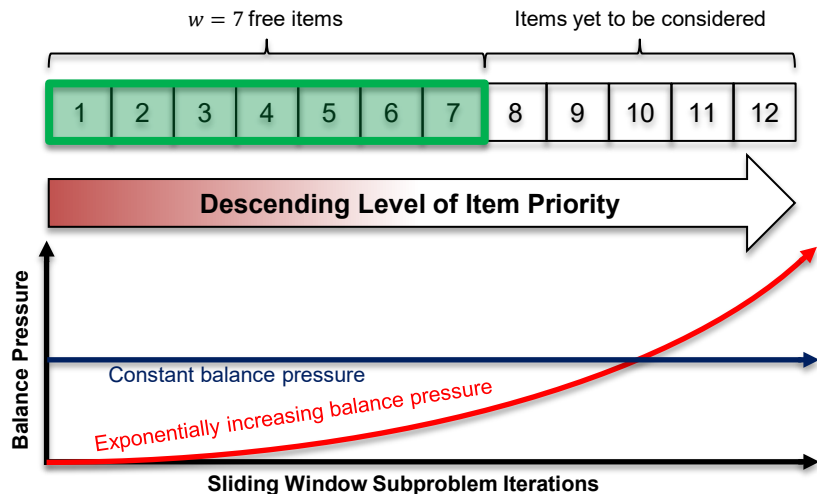
Challenge: Global Constraints in Local Decomposition

Challenge: Load balancing is a global constraint (depends on ALL items), but sliding window decomposition solves local subproblems (window w).

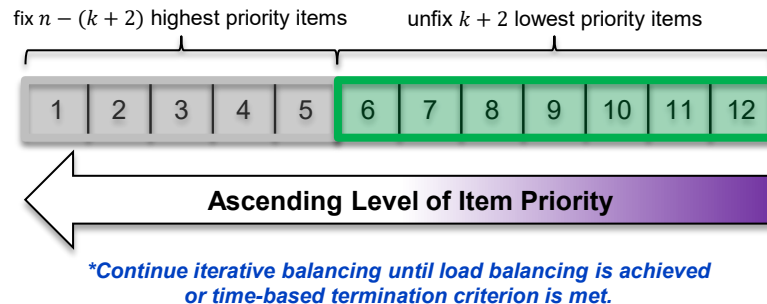
Baseline: Single MILP (with load balancing) works but has lower prioritization scores (previous work).

Goal: Adapt effective sliding window method for load balancing.

In-Stride Balancing



Post-Processing*

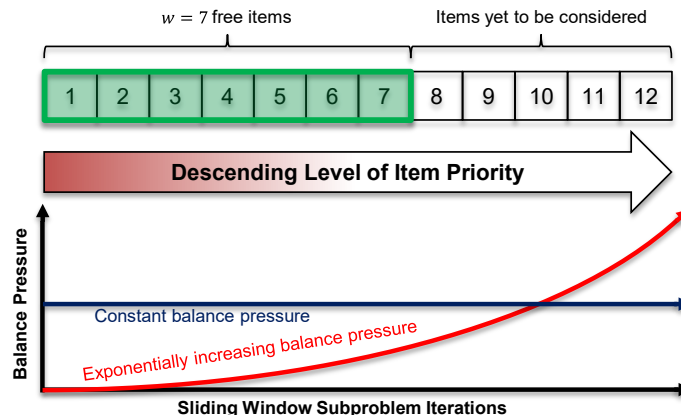


Solution Techniques

In-Stride Load Balancing

Strategy 1: In-Stride Balancing (Penalized Objective):

- **Concept:** Integrate balancing *during* sliding window iterations.
- **Methods:**
 - Constant λ : Fixed penalty on CG deviation.
 - Dynamic λ : Adaptive penalty (e.g., “cold-to-hot” schedule).
- **Penalty Scaling:** Custom grid-based lower bound (GB-LB) used to guide λ magnitude, ensuring consistent trade-off (GB-LB was developed in previous work).



Solution Techniques

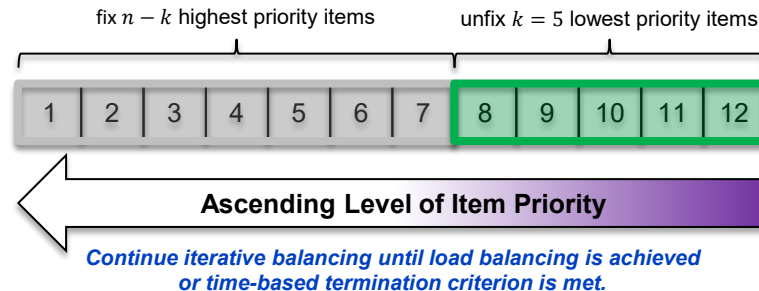
Post-Processing Adjustment for Load Balancing

Strategy 2: Post-Processing Adjustment (Two-Stage):

- **Stage 1: Prioritization Solve:**
 - Run standard sliding window decomposition, optimizing *only* for the prioritization objective.
 - Result: High-quality prioritized layout, but likely unbalanced.
- **Stage 2: Iterative Feasibility-Seeking MILP Adjustment:**
 - **Guiding Principle:** *Systematically increase solution flexibility* from the Stage 1 layout until load-balancing feasibility is achieved. Start with most items fixed, then iteratively unfix coordinates of lowest-priority items (reverse π_{ii} order) and re-optimize.
 - **Goal:** Achieve load balance feasibility while *minimally degrading* prioritization score.

Stage 2 Algorithm: Start with Stage 1 solution (coordinates used for warm start + fixing). Set $k = 1$ (number of free items). While load-balancing is not feasible & k is less than the total number of items:

1. **Identify:** Determine the set of k lowest-priority items.
2. **Define MILP:** Full MILP with load-balancing constraints. Fix coordinates for all items not in the lowest- k set.
3. **Solve MILP:** Use warm start. Apply per-iteration time limit. Use callback for **early termination** if a load-balanced integer solution is found.
4. **Check:** If the final solution from Step 3 satisfies load-balancing constraints → STOP.
5. **Increment:** If not feasible, $k = k + 1$, repeat loop.



Data Generation Strategy

- **Goal:** Create realistic, reproducible military scenario instances.

- **Data Sources:**



JECD (Joint Equipment Characteristic Database): Item dimensions, weight.



TOE (Table of Organizational Equipment) / MTOE (Modified TOE): Unit structures (e.g., Armored Brigade), item quantities).

- **Filtering:** Use EQUIP CODEs to select relevant combat/support vehicles.
- **Groups and Priorities:** Assign items to functional groups (UIC-based); assign plausible tactical priorities.
- **Vessel Specifications:** Define representative Navy/MSV vessel decks (dimensions, target CG, balance tolerance).
- **Instance Generation:** Assign random subsets of groups (ABCT to Division scale) to vessels; consider packing density (item area / bin area ratio).

EQUIPMENT CODE (EQUIP CODE): An alphanumeric character code applied to each vehicle and equipment data listing to enable automated identification and retrieval of selected data categories. Types of equipment codes and their meanings are as follows:

TYPE
EQUIP
CODE

MEANING

Vehicles, wheeled (self-propelled)

- 1 1/2-ton or less
- 2 Sedan
- 3 2-1/2-ton or less
- 4 Greater than 2-1/2-ton
- 5 Materials handling equipment (MHE)
- 8 Construction equipment



Vehicles, wheeled (not self-propelled)

- 6 2-1/2-ton or less
- 7 Greater than 2-1/2-ton
- 9 Construction equipment
- 0 Materials handling equipment (MHE)



Other vehicles

- C Vehicles, tracked or half-tracked (except D and E type equipment codes)
- D Tanks (combat)
- E Artillery (self-propelled)
- F Artillery (towed)
- G Floating craft (except amphibious vehicles that are included under the appropriate numeric code)
- H Aircraft, rotary wing (operational)
- J Aircraft, fixed wing (operational)
- K All aircraft in a reduced configuration
- Nonvehicular and special-handling equipment
- M Class A Explosives
- N Class B Explosives
- P Hazardous items
- Q Ammunition and explosives over .60 caliber
- R Small-arms ammunition .60 caliber or less
- S Yellow TAT organizational equipment and supplies
- T Red TAT organizational equipment and supplies
- U Equipment other than vehicles
- V LVAD (NOTE: Equipment or supplies rigged or packaged for airdrop must be reported as "Special Handling Cargo")
- X Red TAT baggage
- Y Yellow TAT personal baggage

Equipment categorization based on EQUIP CODE used for filtering JECD data.

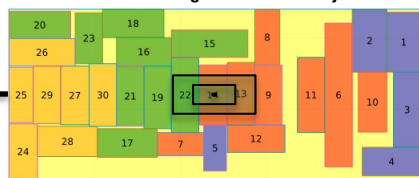
Initial Experiments

Testing Load Balancing Solution Techniques

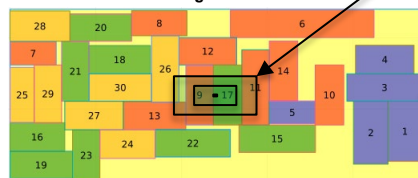
Scope and Intent: Initial tests on 4 diverse instances (e.g., 2-group/15-item to 5-group/55-item) with stringent conditions (1% CG deviation, >75% space utilization) to gain early insights into load balancing efficacy in 'worst-case' scenarios.

10%, 5%, and 1% CG Deviation
Tolerance Boxes

Best: Post-Processing Iterative MILP Adjustment



Single MILP



Performance Results for a 4-Group, 30-Item Instance Sorted by Objective Value

Solution Technique	Objective Value	Load Balanced?	Total Run Time (sec)	Stage 2 Iterations (Post-Processing)	Unpenalized Objective
Iterative MILP Adjustment (Post-Processing)	271404.66	TRUE	125.96	7	NA
Dynamic In-Stride Balancing (Shape = 1.0, Max λ = 0.25)	271676.27	TRUE	127.66	NA	271876.27
Dynamic In-Stride Balancing (Shape = 2.5, Max λ = 0.25)	272913.67	TRUE	144.18	NA	272913.67
Dynamic In-Stride Balancing (Shape = 2.5, Max λ = 0.5)	275074.50	TRUE	152.03	NA	275074.50
Constant In-Stride Balancing (λ = 0.05)	280918.29	TRUE	136.76	NA	280918.29
Dynamic In-Stride Balancing (Shape = 1.0, Max λ = 0.5)	285924.45	TRUE	129.98	NA	285924.45
Single MILP Load Balancing	288849.74	TRUE	150.02	NA	NA
Dynamic In-Stride Balancing (Shape = 5.0, Max λ = 0.25)	305392.78	FALSE	149.73	NA	267184.17
Dynamic In-Stride Balancing (Shape = 5.0, Max λ = 0.5)	308771.73	FALSE	150.38	NA	268449.56
Dynamic In-Stride Balancing (Shape = 10.0, Max λ = 0.25)	334734.92	FALSE	150.14	NA	255731.42
Constant In-Stride Balancing (λ = 0.25)	335598.76	FALSE	84.41	NA	306489.98
Dynamic In-Stride Balancing (Shape = 10.0, Max λ = 0.5)	413051.11	FALSE	150.79	NA	255799.22

- **Feasibility:** Challenging for larger instances; some techniques maintained balance better (e.g., Iterative MILP).
- **Iterative MILP Adjustment:** Consistently robust; often achieved best (or near-best) prioritization score among balanced solutions.
- **Dynamic In-Stride Balancing:** Showed promise, especially with careful λ -schedule tuning; performance varied with problem scale.
- **Single MILP (Benchmark):** Struggled with feasibility and/or solution quality on larger, tightly constrained instances.

Load Balancing Strategies

Key Insights & Refinement Path

Key Insights:

- **Adaptive Strategy Needed:** Optimal load balancing approach varies significantly with problem size and complexity.
- **Post-Processing (Iterative MILP):** Highly robust and promising for achieving balance with good prioritization.
- **In-Stride Parameter Tuning is Critical:** Efficacy of dynamic λ methods hinges on appropriate scheduling (e.g., shape curve parameter, timing of penalty increase).
- **Potential for Hybrid Approaches:** In-stride methods getting 'close' to balance could benefit from lightweight post-processing.

Planned Further Investigation:

- Systematic tuning of dynamic λ scheduling parameters for in-stride methods.
- Investigating targeted post-processing for nearly-balanced in-stride solutions.
- Exploring alternative Stage 2 unfixing strategies for Iterative MILP adjustment (e.g., based on item weight, size, or initial placement).

Path Forward

Impact & Conclusion

Full Experimentation: Building on initial insights, a comprehensive study (based on the discussed data generation strategy) will rigorously evaluate and refine these load balancing strategies across diverse, realistic scenarios.

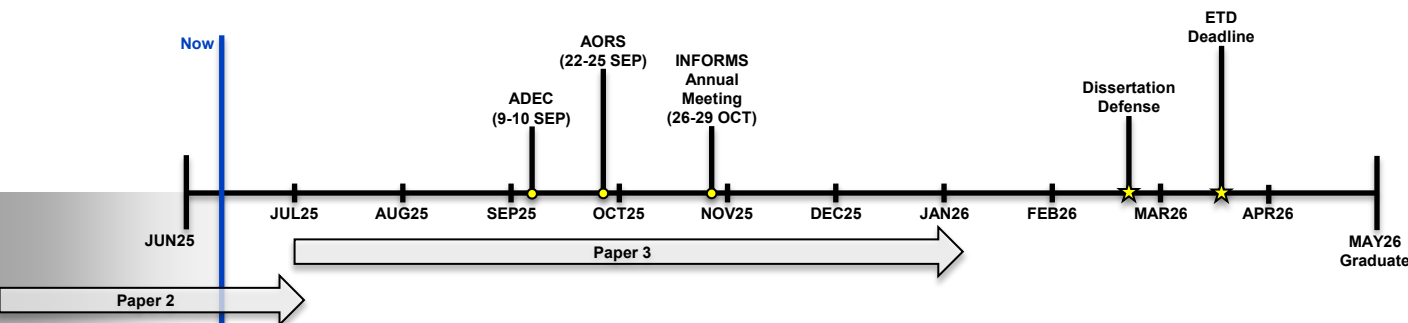
Expected Contributions:

- Novel framework for prioritized military load planning with integration of critical operational constraints.
- Effective decomposition strategies for a complex, constrained OR problem.

Core Takeaway: Automating Combat Load Planning.

- **Automated & Optimized:** This research delivers a method for automatically generating load plans that are both operationally feasible (e.g., balanced) and strategically optimized for multi-level tactical priorities.
- **Systematizing Commander's Intent:** It provides a robust capability to embed complex, commander-driven requirements directly into the planning process, crucial for enhancing readiness in modern contested environments.

Future: Multi-vessel load planning (Next Dissertation Chapter – Paper 3).



William Kirschenman
PhD Candidate - Operations Research at North
Carolina State University

